

Exercise 9: Multivariate Optimization 4 (Newton-Raphson + Gauss-Newton)

Optimization in Machine Learning

Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

Python

```
pip install numpy==2.0.2 matplotlib==3.10.8
```

R

```
install.packages("remotes")  
remotes::install_version("ggplot2", "3.5.1")  
remotes::install_version("gridExtra", "2.3")
```

Setup

We model the data with $\pi : \mathbb{R}^2 \rightarrow [0, 1]$ defined as

$$\pi(x_1, x_2) = \frac{1}{1 + \exp((1, x_1, x_2)^\top \theta)}$$

and minimize the **Brier loss**

$$\mathcal{R}_{\text{emp}} = \sum_{i=1}^n \|y^{(i)} - \pi(\mathbf{x}^{(i)})\|_2^2.$$

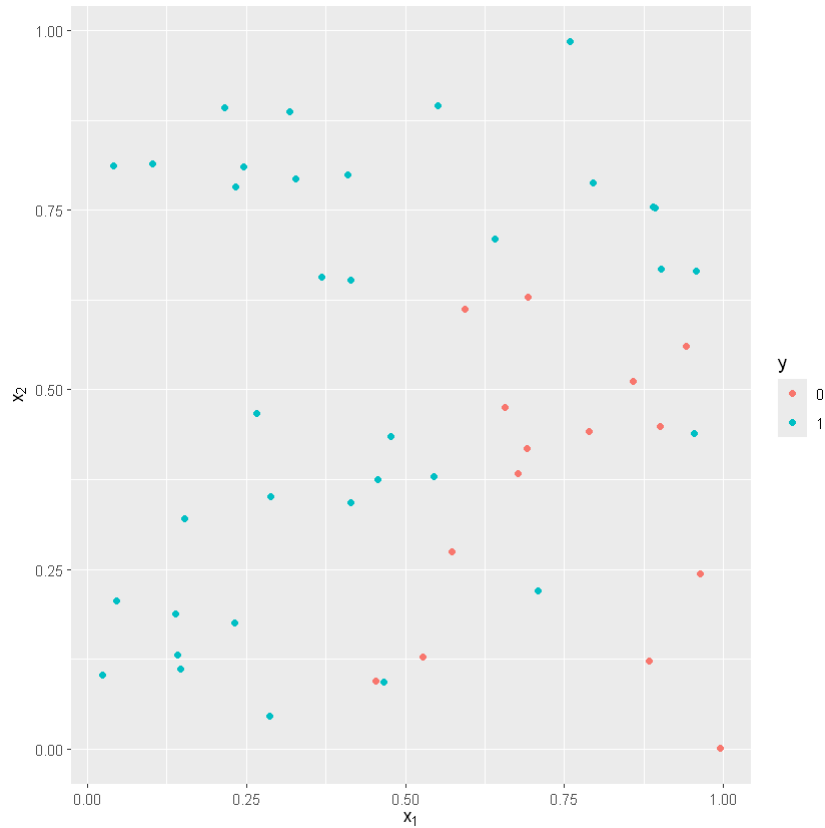
Note the **flipped sigmoid** convention: $\pi(x) \rightarrow 0$ as $\tilde{\mathbf{x}}^\top \theta \rightarrow +\infty$ (the opposite of the standard logistic). This is a quirk of this exercise's parameterization — keep it in mind when reading gradient signs below.

R

```
library(ggplot2)

set.seed(123)
n <- 50
X <- matrix(runif(2 * n), ncol = 2)
X_model <- cbind(1, X) # n x 3 design
y <- -((X_model %*% c(0.3, -1, 1) + rnorm(n, 0, 0.3) < 0) - 1)
df <- as.data.frame(X); df$type <- as.character(y)

ggplot(df) +
  geom_point(aes(x = V1, y = V2, color = type)) +
  xlab(expression(x[1])) + ylab(expression(x[2])) +
  labs(color = "y")
```



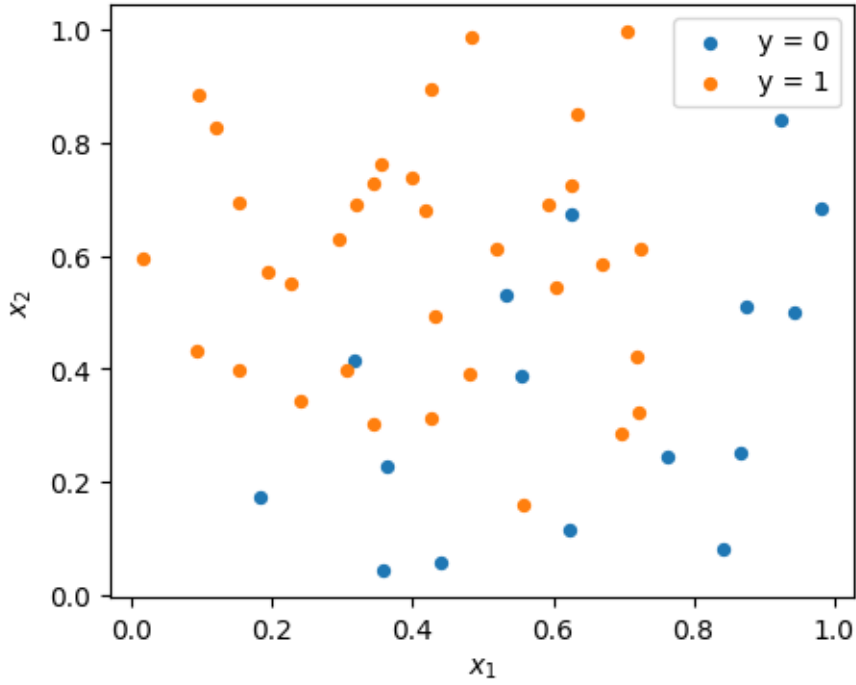
Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(123)
n = 50
X = np.random.uniform(size=(n, 2))
X_model = np.column_stack([np.ones(n), X]) # n x 3 design
lin = X_model @ np.array([0.3, -1.0, 1.0]) + np.random.normal(0, 0.3, n)
y = -((lin < 0).astype(int) - 1)
y = y.astype(float)

fig, ax = plt.subplots(figsize=(5, 4))
for label in (0, 1):
    mask = y == label
    ax.scatter(X[mask, 0], X[mask, 1], label=f"y = {label}", s=18)
```

```
ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$")
ax.legend()
plt.show()
```



i Note

R vs. Python parity. R's `set.seed(123)` and Python's `np.random.seed(123)` use *different* RNGs and call them in different orders, so the simulated data differ between languages. NR and GN are deterministic given the data, so the *trajectories* (as printed below) will not match between languages — but the *qualitative behavior* (NR converging quadratically, GN converging more slowly) should match.

(a) Gradient of the Brier loss

Show that

$$\nabla_{\theta} \mathcal{R}_{\text{emp}} = \sum_{i=1}^n 2 \cdot \frac{y^{(i)} \exp(f^{(i)}) - (\exp(-f^{(i)}) + 1)^{-1}}{(\exp(f^{(i)}) + 1)^2} \cdot \tilde{\mathbf{x}}^{(i)}$$

where $\tilde{\mathbf{x}}^{(i)} = (1, x_1^{(i)}, x_2^{(i)})^\top$ and $f^{(i)} = \tilde{\mathbf{x}}^{(i)\top} \theta$.

Solution.

Define $s : \mathbb{R} \rightarrow \mathbb{R}, f \mapsto \frac{1}{1+\exp(f)}$.

$$\begin{aligned}
\nabla_{\theta} \mathcal{R}_{\text{emp}} &= \nabla_{\theta} \sum_{i=1}^n \|y^{(i)} - f(\mathbf{x}^{(i)})\|_2^2 = \sum_{i=1}^n \frac{d}{df} \|y^{(i)} - s(f^{(i)})\|_2^2 \cdot \nabla_{\theta} f^{(i)} \\
&= \sum_{i=1}^n 2 \frac{y^{(i)}(\exp(f^{(i)}) + 1) - 1}{\exp(f^{(i)}) + 1} \cdot \frac{\exp(f^{(i)})}{(\exp(f^{(i)}) + 1)^2} \tilde{\mathbf{x}}^{(i)} \\
&= \sum_{i=1}^n 2 \frac{y^{(i)}(\exp(f^{(i)}) + 1) - 1}{\exp(f^{(i)}) + 1} \cdot \frac{\frac{\exp(f^{(i)})}{\exp(f^{(i)}) + 1}}{\exp(f^{(i)}) + 1} \tilde{\mathbf{x}}^{(i)} \\
&= \sum_{i=1}^n 2 \frac{y^{(i)}(\exp(f^{(i)})) - \frac{\exp(f^{(i)})}{\exp(f^{(i)}) + 1}}{(\exp(f^{(i)}) + 1)^2} \tilde{\mathbf{x}}^{(i)} \\
&= \sum_{i=1}^n 2 \frac{y^{(i)}(\exp(f^{(i)})) - (\exp(-f^{(i)}) + 1)^{-1}}{(\exp(f^{(i)}) + 1)^2} \tilde{\mathbf{x}}^{(i)}
\end{aligned}$$

(b) Hessian of the Brier loss

Show that

$$\nabla_{\theta}^2 \mathcal{R}_{\text{emp}} = \sum_{i=1}^n 2 \cdot \frac{\exp(f^{(i)})(y^{(i)}(1 - \exp(2f^{(i)})) - 1 + 2 \exp(f^{(i)}))}{(\exp(f^{(i)}) + 1)^4} \cdot \tilde{\mathbf{x}}^{(i)} \tilde{\mathbf{x}}^{(i)\top}.$$

Solution.

Introduce the shorthand $E := \exp(f^{(i)})$ (so $\exp(-f^{(i)}) = E^{-1}$) and drop the per-sample index i inside the sum ($y := y^{(i)}, \tilde{\mathbf{x}} := \tilde{\mathbf{x}}^{(i)}$). Differentiating the gradient from (a):

$$\begin{aligned}
\nabla_{\theta}^2 \mathcal{R}_{\text{emp}} &= \sum_{i=1}^n \frac{d}{df} 2 \frac{yE - (E^{-1} + 1)^{-1}}{(E + 1)^2} \tilde{\mathbf{x}} \nabla_{\theta} f^{\top} \\
&= \sum_{i=1}^n 2 \frac{(yE - (E^{-1} + 1)^{-2} E^{-1})(E + 1)^2 - (yE - (E^{-1} + 1)^{-1}) 2(E + 1)E}{(E + 1)^4} \tilde{\mathbf{x}} \tilde{\mathbf{x}}^{\top} \\
&= \sum_{i=1}^n 2 \frac{yE(E + 1)^2 - E - (2yE(E + 1) - 2E)E}{(E + 1)^4} \tilde{\mathbf{x}} \tilde{\mathbf{x}}^{\top} \\
&= \sum_{i=1}^n 2 \frac{E(y(E + 1)^2 - 1 - 2yE(E + 1) + 2E)}{(E + 1)^4} \tilde{\mathbf{x}} \tilde{\mathbf{x}}^{\top} \\
&= \sum_{i=1}^n 2 \frac{E(y(E^2 + 2E + 1 - 2E^2 - 2E) - 1 + 2E)}{(E + 1)^4} \tilde{\mathbf{x}} \tilde{\mathbf{x}}^{\top} \\
&= \sum_{i=1}^n 2 \frac{E(y(1 - E^2) - 1 + 2E)}{(E + 1)^4} \tilde{\mathbf{x}} \tilde{\mathbf{x}}^{\top}
\end{aligned}$$

(c) Non-convexity

Show that $\mathcal{R}_{\text{emp}}$ is not convex in general.

Solution.

Assume, e.g., there is only one observation with $y^{(1)} = 0$ then

$$\nabla_{\theta}^2 \mathcal{R}_{\text{emp}} = \frac{2 \exp(f^{(1)})(2 \exp(f^{(1)}) - 1)}{(\exp(f^{(1)}) + 1)^4} \underbrace{\tilde{\mathbf{x}}^{(1)} \tilde{\mathbf{x}}^{(1)\top}}_{\text{p.s.d.}}.$$

If a p.s.d. matrix is multiplied with a negative number it becomes a n.s.d. matrix, i.e., $\nabla_{\theta}^2 \mathcal{R}_{\text{emp}}$ is n.s.d. if $2 \exp(f^{(1)}) < 1 \iff f^{(1)} < \ln(0.5)$. This condition trivially holds, e.g., if $\theta = (\ln(0.5) - 1, 0, 0)^{\top}$.

(d) Newton-Raphson

Implement Newton-Raphson to minimize $\mathcal{R}_{\text{emp}}$. Run for 30 iterations starting at $\theta^{[0]} = \mathbf{0}$. Plot $\mathcal{R}_{\text{emp}}$ and the parameter trajectory.

Solution.

R

```
# Brier loss + gradient + Hessian using named intermediates from solution (b):
# E = exp(f), D = E + 1, s = 1/D = pi(x).
remp <- function(theta) {
  E <- exp(X_model %>% theta)
  sum((y - 1 / (E + 1))^2)
}

grad_remp <- function(theta) {
  E <- exp(X_model %>% theta)
  inner <- 2 * (y * E - E / (E + 1)) / (E + 1)^2      # per-sample d_R/d_f
  c(t(inner) %>% X_model)                             # chain rule x df/dtheta
}

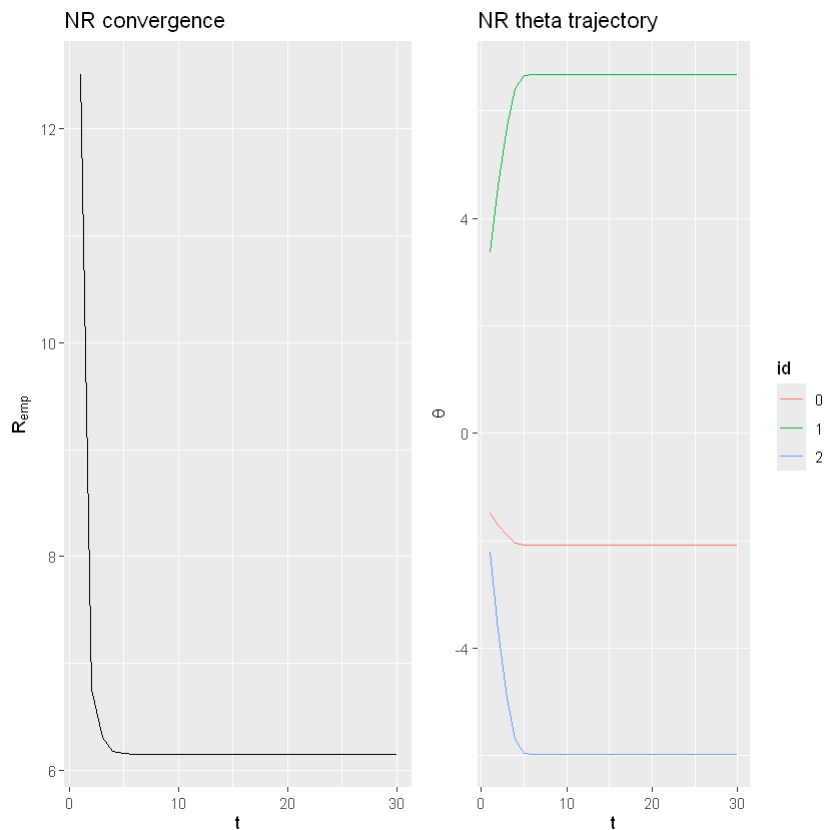
hess_remp <- function(theta) {
  E <- exp(X_model %>% theta)
  diag_factor <- as.numeric(2 * E * (y * (1 - E^2) - 1 + 2 * E) / (E + 1)^4)
  t(X_model) %>% (diag_factor * X_model)
}

# Newton-Raphson loop
theta <- c(0, 0, 0)
remps_nr <- numeric(30)
thetas_nr <- matrix(NA, 30, 3)
for (i in 1:30) {
  remps_nr[i] <- remp(theta)
  theta <- theta + solve(hess_remp(theta), -grad_remp(theta))
  thetas_nr[i, ] <- theta
}
cat(sprintf("NR final theta = (%+.4f, %+.4f, %+.4f), R_emp = %.4f\n",
            theta[1], theta[2], theta[3], remp(theta)))

# Diagnostic plots
library(gridExtra)
p1 <- ggplot(data.frame(t = 1:30, R = remps_nr), aes(t, R)) + geom_line() +
  ylab(expression(R[emp])) + ggtitle("NR convergence")
p2 <- ggplot(data.frame(t = rep(1:30, 3),
                        theta = c(thetas_nr),
                        id = factor(rep(0:2, each = 30)))) +
  geom_line(aes(t, theta, color = id)) +
  ylab(expression(theta)) + ggtitle("NR theta trajectory")
```

```
grid.arrange(p1, p2, ncol = 2)
```

NR final theta = (-2.0871, +6.6674, -5.9675), R_emp = 6.1561



Python

```
# Brier loss + gradient + Hessian using named intermediates from solution (b):  
#  $E = \exp(f)$ ,  $D = E + 1$ ,  $s = 1/D = \pi(x)$ .  
def remp(theta):  
    E = np.exp(X_model @ theta)  
    return float(np.sum((y - 1 / (E + 1)) ** 2))  
  
def grad_remp(theta):  
    E = np.exp(X_model @ theta)
```

```

inner = 2 * (y * E - E / (E + 1)) / (E + 1) ** 2      # per-sample d_R/d_f
return X_model.T @ inner

def hess_remp(theta):
    E = np.exp(X_model @ theta)
    diag_factor = 2 * E * (y * (1 - E ** 2) - 1 + 2 * E) / (E + 1) ** 4
    return X_model.T @ (diag_factor[:, None] * X_model)

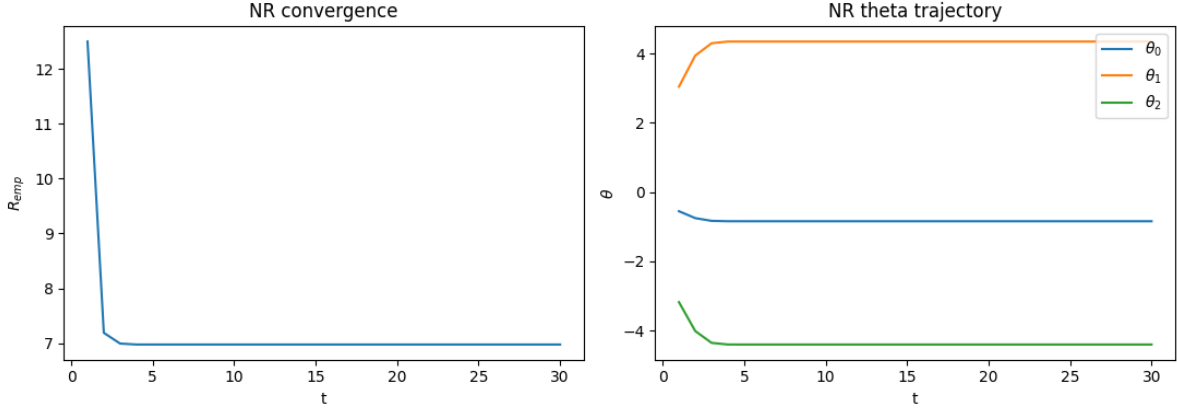
theta = np.zeros(3)
remps_nr = np.empty(30)
thetas_nr = np.empty((30, 3))
for i in range(30):
    remps_nr[i] = remp(theta)
    theta = theta + np.linalg.solve(hess_remp(theta), -grad_remp(theta))
    thetas_nr[i] = theta

print(f"NR final theta = ({theta[0]:+.4f}, {theta[1]:+.4f}, {theta[2]:+.4f}), "
      f"R_emp = {remp(theta):.4f}")

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].plot(np.arange(1, 31), remps_nr)
axes[0].set_xlabel("t"); axes[0].set_ylabel(r"$R_{\text{emp}}$")
axes[0].set_title("NR convergence")
for j in range(3):
    axes[1].plot(np.arange(1, 31), thetas_nr[:, j], label=fr"$\theta_{\text{j}}$")
axes[1].set_xlabel("t"); axes[1].set_ylabel(r"$\theta$"); axes[1].legend()
axes[1].set_title("NR theta trajectory")
plt.tight_layout()
plt.show()

```

NR final theta = (-0.8428, +4.3518, -4.4067), R_emp = 6.9744



For Newton-Raphson, we need to solve in each update step

$$\nabla_{\theta}^2 \mathcal{R}_{\text{emp}} \mathbf{d} = -\nabla_{\theta} \mathcal{R}_{\text{emp}}$$

for the descend direction $\mathbf{d}$.

(e) Gauss-Newton

Explain why Gauss-Newton is applicable here, and implement it (30 iterations, $\theta^{[0]} = \mathbf{0}$). Compare its convergence to Newton-Raphson from (d).

Solution.

In this case, we can apply Gauss-Newton since $\mathcal{R}_{\text{emp}}$ is the squared sum of the residuals

$$\mathbf{r} = (y^{(1)} - \pi(\mathbf{x}^{(1)}), \dots, y^{(n)} - \pi(\mathbf{x}^{(n)}))^{\top}.$$

Here, for the update step we need to compute

$$\nabla_{\theta} \mathbf{r} = \begin{pmatrix} \frac{\exp(f^{(1)})}{(1+\exp(f^{(1)}))^2} \tilde{\mathbf{x}}^{(1)\top} \\ \vdots \\ \frac{\exp(f^{(n)})}{(1+\exp(f^{(n)}))^2} \tilde{\mathbf{x}}^{(n)\top} \end{pmatrix}$$

For Gauss-Newton, we solve in each update step

$$(\nabla_{\theta} \mathbf{r}^{\top} \nabla_{\theta} \mathbf{r}) \cdot \mathbf{d} = -\nabla_{\theta} \mathbf{r}^{\top} \cdot \mathbf{r}$$

for the descend direction $\mathbf{d}$.

R

```

options(repr.plot.width = 11, repr.plot.height = 4)
# Gauss-Newton: replace Hessian with J^T J where J is the residual Jacobian.
# residual r_i = y_i - pi(x_i),
# dr_i/dtheta = -pi'(f) * x_i = (E / (E+1)^2) * x_i.
theta <- c(0, 0, 0)
remps_gn <- numeric(30)
thetas_gn <- matrix(NA, 30, 3)
for (i in 1:30) {
  remps_gn[i] <- remp(theta)
  E <- exp(X_model %*% theta)
  r <- y - 1 / (E + 1) # residuals
  J <- as.numeric(E / (E + 1)^2) * X_model # residual Jacobian
  theta <- theta + solve(t(J) %*% J, -t(J) %*% r)
  thetas_gn[i, ] <- theta
}
cat(sprintf("GN final theta = (%+.4f, %+.4f, %+.4f), R_emp = %.4f\n",
            theta[1], theta[2], theta[3], remp(theta)))

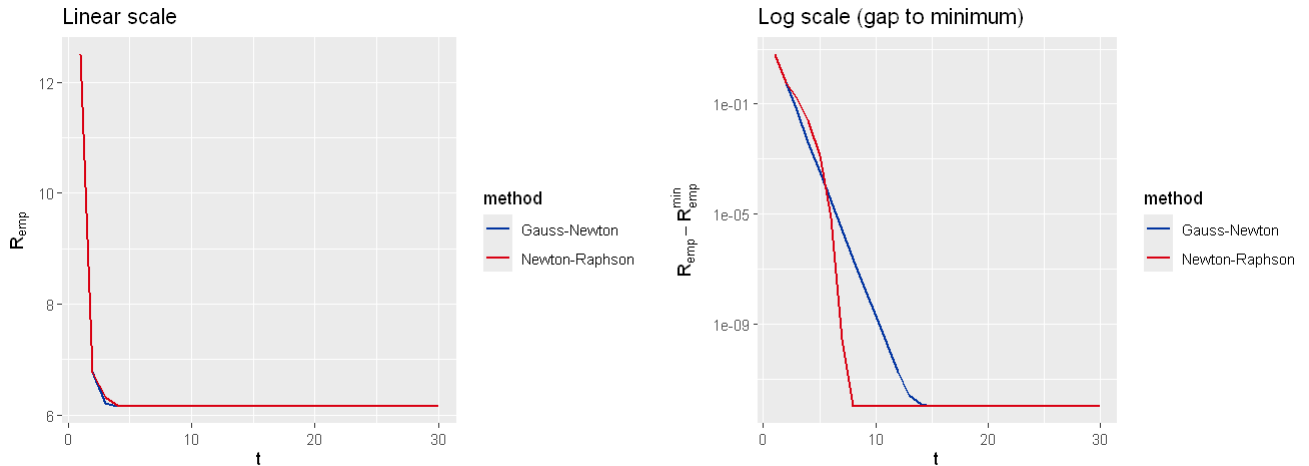
# Compare NR vs GN convergence. Linear axis (left) hides the convergence-rate
# distinction once both are close to the minimum; log-axis on R_emp - R_emp_min
# (right) exposes it.
R_min <- min(remps_nr, remps_gn) - 1e-12
df_compare <- rbind(
  data.frame(t = 1:30, R = remps_nr, gap = remps_nr - R_min,
             method = "Newton-Raphson"),
  data.frame(t = 1:30, R = remps_gn, gap = remps_gn - R_min,
             method = "Gauss-Newton")
)

p_lin <- ggplot(df_compare, aes(t, R, color = method)) +
  geom_line(linewidth = 0.7) +
  scale_color_manual(values = c("Newton-Raphson" = "#D90012",
                                "Gauss-Newton" = "#0033A0")) +
  ylab(expression(R[emp])) + ggtitle("Linear scale")
p_log <- ggplot(df_compare, aes(t, gap, color = method)) +
  geom_line(linewidth = 0.7) +
  scale_y_log10() +
  scale_color_manual(values = c("Newton-Raphson" = "#D90012",
                                "Gauss-Newton" = "#0033A0")) +
  ylab(expression(R[emp] - R[emp]^{min})) +

```

```
ggtitle("Log scale (gap to minimum)")
grid.arrange(p_lin, p_log, ncol = 2)
```

GN final theta = (-2.0871, +6.6674, -5.9675), R_emp = 6.1561



Python

```
# Gauss-Newton: replace Hessian with  $J^T J$ . Residual  $r_i = y_i - \pi(x_i)$ ,
#  $dr_i/d\theta = -\pi'(f) * x_i = (E / (E+1)^2) * x_i$ .
theta = np.zeros(3)
remps_gn = np.empty(30)
thetas_gn = np.empty((30, 3))
for i in range(30):
    remps_gn[i] = remp(theta)
    E = np.exp(X_model @ theta)
    r = y - 1 / (E + 1) # residuals
    J = (E / (E + 1) ** 2)[:, None] * X_model # residual Jacobian
    theta = theta + np.linalg.solve(J.T @ J, -J.T @ r)
    thetas_gn[i] = theta

print(f"GN final theta = ({theta[0]:+.4f}, {theta[1]:+.4f}, {theta[2]:+.4f}), "
      f"R_emp = {remp(theta):.4f}")

# Compare NR vs GN convergence. Linear axis (left) hides the convergence-rate
# distinction once both are close to the minimum; log-axis on R_emp - R_emp_min
```

```

# (right) exposes it.
R_min = min(remps_nr.min(), remps_gn.min()) - 1e-12
iters = np.arange(1, 31)

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].plot(iters, remps_nr, color="#D90012", label="Newton-Raphson")
axes[0].plot(iters, remps_gn, color="#0033A0", label="Gauss-Newton")
axes[0].set_xlabel("t"); axes[0].set_ylabel(r" $R_{\text{emp}}$ ")
axes[0].set_title("Linear scale"); axes[0].legend()

axes[1].semilogy(iters, remps_nr - R_min, color="#D90012",
                 label="Newton-Raphson")
axes[1].semilogy(iters, remps_gn - R_min, color="#0033A0", label="Gauss-Newton")
axes[1].set_xlabel("t"); axes[1].set_ylabel(r" $R_{\text{emp}} - R_{\text{emp}}^{\text{min}}$ ")
axes[1].set_title("Log scale (gap to minimum)"); axes[1].legend()
plt.tight_layout()
plt.show()

```

GN final theta = (-0.8428, +4.3518, -4.4067), $R_{\text{emp}} = 6.9744$

